

Molecular Dynamics Simulations on FPGA with High-Bandwidth Memory
First Last^{1,5,✉}, First Middle Last^{1,2,5}, First M.M. Last^{2,3}, First Last, Jr.^{3,*}, and F. Middle
Last^{3,4,6,7,8,**}

¹University A, London SW7 2AZ, UK

²Department B, University B, Toronto, ON M5S 3H6, Canada

³Division C, Department C, University C, Cambridge, MA, USA

⁴Present address: 123 Sesame Street, New York, NY, USA

⁵These authors contributed equally

⁶Senior author

⁷Twitter: @CorrespondingAuthor

⁸Lead contact

*Correspondence: first.last.jr@university.edu

**Correspondence: f.middle.last@university.edu

SUMMARY

Molecular dynamics (MD) simulation is a powerful tool that models the behavior of particles over a given time interval, complementing biological and chemical experimental methods, particularly in materials science and drug discovery. The speed of MD simulations is one of the key challenges that prevent their applications. To address this, a full hardware module based on field programmable gate arrays (FPGAs) has been designed to accelerate all parts of MD simulations. The solution aims to increase computational speed by utilizing pipeline techniques (Pipeline) to optimize the allocation of individual tasks within the entire parallel system. We have also explored the impact of high-bandwidth memory (HBM2) on computational performance, comparing it with existing memory structures. The optimized design of the underlying register transfer level (RTL) circuitry for a single node has fundamentally enhanced the efficiency of data read and write operations to and from registers. We present the co-design of hardware-software co-optimized infrastructure leveraging a hybrid CPU/FPGA architecture specifically engineered to accelerate MD simulations using the classical CHARMM36 force field. The implementation of our work was demonstrated by comparing the simulation results of noble gas argon and dihydrofolate reductase (DHFR). Additionally, potential strategies for continuing to improve the efficiency of MD simulations are also discussed.

KEYWORDS

Molecular Dynamics, FPGA, PME, accelerate,

INTRODUCTION

Due to its ability to provide detailed mechanistic insights into various physical interactions at atomic level, MD simulations have shown paramount importance for driving the rational discovery of therapeutic agents and functional materials¹⁻⁴. However, the functionally-relevant processes are mostly driven by the motions of aggregated ensemble of molecules, which feature large spatial scales that are computationally demanding to model comprehensively. Consider the many body systems interacting through classical pairwise potentials, the computational demand for integrating the equation of motions quickly escalates with the number of interacting particles in the simulation. Moreover, most of the biophysical activated processes that are of mechanistic interests are known to feature

heterogeneous timescales.⁵ For example, essential cellular activities fundamental to life, such as active solute transports⁶, could take place at the sub-seconds scale, while the vibrational motions of the molecules is characterized on febo-to-picooseconds. The multi-scale nature of the simulated system limits the length of time steps to be sufficiently short (typically 0.5 to 2 fecoseconds) for stable dynamical integration, rendering the computational modeling of long time collective behaviors of the molecules highly intensive.⁷ Aiming to extend the accessible simulation scales subjected to modern computation hardware, contemporary mainstream MD implementations have incorporated multi-CPU parallel strategies and GPU-based codes.⁸⁻¹² However, even with GPU acceleration, the reported trajectory lengths in present MD studies merely exceed several microseconds, and numerically solving the molecular trajectories by MD could cost months of fully loaded computational power for systems containing a few million particles.¹³

Beyond conventional CPU/GPU-based solutions, heterogeneous computing architectures have emerged as a promising paradigm for MD simulations. Recent implementations integrate microcontroller units (MCUs), field-programmable gate arrays (FPGAs), and complex programmable logic devices (CPLDs) through dedicated photonic interconnects, exemplified by the MD-GRAPE cluster architecture that achieves sub-microsecond communication latency between processing nodes.^{14,15} The commercial ANTON supercomputer series utilized the Application Specific Integrated Circuit (ASIC) process to construct a specifically designed cluster with 64 or 512 nodes.¹⁶⁻¹⁸ However, these architectures are commercially expensive to implement, which limits their applicability for routine computations. On the other hand, the Novo-G reconfigurable computing design combines traditional CPUs with programmable hardware (such as FPGAs) to tackle the energy efficiency and performance bottlenecks, and have shown prominent potential for MD simulations.^{19,20}

Compared to the alternative computing hardware, FPGAs offer greater design flexibility and higher environmental friendlies with competitive computational efficiency.²¹⁻²⁴ FPGAs incorporate a variety of unique production techniques, such as the flash process and the reverse fusion process, which allow them to support multiple reconfigurations and debugs in the commercial sector, accommodating different experimental design requirements.²⁵ In the chip design phase, FPGA design is considered the core of prototyping, indicating that FPGA circuit design has significant potential for practical application in the later stages of conversion into an ASIC process.^{26,27} With their high parallel computing capability, reconfigurability, and excellent energy efficiency ratio, FPGAs exhibit unique advantages in MD simulations. They can significantly accelerate intensive floating point computations by processing particle interaction forces, such as van der Waals forces and Coulomb forces, in parallel through thousands of configurable logic units. The hardware pipeline design of FPGAs enables efficient coordination of adjacent list generation, force calculation, and integration processes. Moreover, FPGAs can flexibly adapt to different potential functions in MD simulation and various numerical integration methods thereby meeting the diverse requirements from rigid body to coarse-grained models.²⁸ FPGA device fabrication technology has been undergoing rapid advancements, particularly with process nodes advancing to 7nm and below, enabling unprecedented performance improvements in high-performance computing applications. The emergence of HBM2 memory (2.5D TSV-stacked HBM2e achieves 5.3 TB/s bandwidth with $8.5\times$ lower latency than DDR5) addresses the critical data-bandwidth bottleneck in molecular dynamics (MD) simulations²⁹, where nanosecond-scale force calculations require >100 GB/s sustained memory throughput to sustain simulation accuracy. Despite these advancements, no existing FPGA-based MD implementations have yet leveraged HBM2's unique combination of bandwidth-density and near-memory compute capabilities, leaving a critical technology-to-physics gap in scaling MD simulations beyond 10-million-atom systems.

RESULTS

0.1 Experimental Results of the Particle Pair Computing Module Designed via HLS

The HLS compilation tool can optimize DSP resources, and we focus on testing the performance optimization effect of particle-to-computation units, using the HLS synthesis tool for first-order linear interpolation. As shown in Figure 1, using Xilinx’s HLS for the multiply-add implementation of second-order polynomials, the synthesis results in the use of 4 floating-point IPs (2 multipliers and 2 adders) for a total of 10 DSP48 modules.³⁰ With a clock constraint of 300 MHz, the pipeline optimization requires 22 clock cycles to complete the implementation. If the clock constraint is 200 MHz, it takes 18 clock cycles for pipeline to complete the function. To ensure the accuracy of the calculation and consider the range of values of the coefficient table, it is not worthwhile to convert the fixed-point realization. Regardless of the implementation method, if Xilinx’s floating-point computing IP is ultimately used, the performance and resource usage can be expected to be comparable.³¹ The calculation of the r_{ij} consumes approximately 19 DSP48 modules and takes 25 clock cycles. Each RL module uses approximately one r_{ij}^2 computational unit for particle pair screening, three computational units for interpolating polynomials and one force merger accumulator for force vector computation, and a number of accumulators for accumulating force vectors of neighboring cells.

Table 1: Performance and resource evaluation of multiplication and addition of first-order polynomials using Xilinx’s HLS, totaling 10 DSP48 modules. With a clock constraint of 300 MHz, Pipeline requires 22 clock cycles to complete the functional implementation. The performance and resource evaluation of multiply-add second order polynomials using Xilinx’s HLS is performed on a total of 16 DSP48 modules. Pipeline requires 34 clock cycles to complete the functional implementation; Xilinx’s HLS is used to evaluate the performance and resources of the multiply-add operation for third-order polynomials, totaling 22 DSP48 modules. With a clock constraint of 300 MHz, Pipeline requires 45 clock cycles to complete the function

Modules	Latency (cycles)	Latency (ns)	DSP	FF	LUT
First Order Linear Interpolation	22	73.304	10	1459	763
Second Order Linear Interpolation	34	113.288	16	1788	1063
Third Order Linear Interpolation	45	148.514	22	2442	1452

0.2 Resource Consumption of the Computing Units in Several Main Modules

This study focuses on the core challenges of using FPGA to accelerate MD simulations. Aiming at the most time-consuming part, i.e., the non-bonded force calculation, a system optimization scheme based on a heterogeneous computing architecture is proposed. As shown in Table 3, by comparing the performance characteristics of general-purpose computing and dedicated hardware architectures, the technical advantages of FPGA in meeting future high computing power requirements are established. The following table presents the resource consumption of the limited range force module and that of the double limited range force modules. As shown in Table 2, in the future computing platform, these components are to be assembled into a large scale computing matrix. Regarding the Coulomb force, it shows the resource consumption of the long - range force calculation module, which makes use of the Xilinx FFT 9.0 calculation IP.³² Also included is the resource consumption of the bonded force module.

Table 2: Comparison of resources used by a single module and two RL modules in finite range. The FPGA platform was Xilinx Virtex UltraScale+ VU37P with Inspur F37X.

Type	Moldule 2 utilization rate %	Moldule 2 utilization rate %
LUT	10	21
LUTRAM	3	6
FF	13.13	26.4
BRAM	7	18.5
URAM	0.63	~
DSP	2	4
IO	1	1
GT	17	34
BUFG	3	6
MMCM	8	16
PLL	17	34

Table 3: Layout routing uses Xilinx Virtex UltraScale+ VU37P chip, FFT 64-bit sequence resource utilization based on URAM/BRAM bar chart. Resource utilization rates for covalent bond module implemented on Xilinx Virtex UltraScale+ VU37P chip through layout routing.

Resource Name	Utilization %		
	URAM	BRAM	BOND
LUT	15	16	15
LUTRAM	4	4	4
FF	14	14	14
BRAM	24	58	24
URAM	13	-	13
DSP	7	7	3
IO	1	1	1
GT	17	17	17
BUFG	2	2	2
MMCM	8	8	8
PCIe	17	17	17

0.3 Experimental Results of the Fourier Transform Module

In the calculation of PME, the speed of 3D FFT calculation needs to be improved. We use FFT 9.0 IP to optimize the computing architecture to improve the computing efficiency. In the FFT 9.0 user manual³³, the pipelined mode offers the fastest speed but increases resource consumption exponentially as shown in Table 4.

The 3D FFT calculation needs to calculate the data of the 3D XYZ separately. The data at the end of one dimension calculation is the data at the beginning of the next dimension transformation. The specific architecture is shown in the Figure 12. After completing FFT computations, the FFT module transfers data to the data processing module. If Z-dimension FFTs are completed, the convolution module retrieves coefficients from the coefficient matrix local memory for convolution; other dimensions skip convolution. Processed data is then sent to the data conversion module.

Table 4: Implementation of 3D FFT computation module using Xilinx Virtex UltraScale+ VU37P accelerator card. The module completes data reading, 3D FFT computation, convolution, inverse transform, and data storage. Pipelined mode is configured for testing. Input data includes single-precision floating-point matrices of dimensions $32 \times 32 \times 32$ and $64 \times 64 \times 64$.

	Model	Clock Frequency	$32 \times 32 \times 32$ (μs)	$64 \times 64 \times 64$ (μs)
FPGA	VU37P	300 MHz	18.33	119.7
NVIDIA	Tesla K20c	0.73 GHz	29	92
Anton			4	13

0.4 Verification of LJ Particle Energy and Forces

To validate our design and implementation for calculating forces in force field based particle simulations on FPGAs, we conducted a series of tests. We first verified the LJ calculations using the simplest validation system: a two particle LJ system with parameters $\sigma = 3.0 \text{ \AA}$ and $\varepsilon = 0.5 \text{ kcal/mol}$. Since the goal of this test is to verify the accuracy of LJ term computation rather than simulate practical scenarios, the parameters is arbitrary.

We calculated the system’s energy and the force on one particle (the two particles being identical, verifying one suffices). The simulation conditions included a $30 \text{ \AA}$ periodic box with a cutoff distance of $15 \text{ \AA}$. Energies and forces were computed for inter-particle distances ranging from 2 to $28 \text{ \AA}$. For hardware, we used an Intel CPU i5-8400 (2.80 GHz base frequency) with OpenMM version 6.0.0, chosen due to its readable 3D Fourier transform implementation in early versions.^{34,35} Tests confirmed consistent results between OpenMM 6.0 and the current 8.0 version. Compilation was performed using GNU C/C++ compiler version 13.3.0. The GPU platform comprised an NVIDIA GeForce GTX 1650 with driver version 550.120 and CUDA 12.4, utilizing mixed-precision computation. All subsequent sections default to these hardware and testing environments unless specified.

The results, shown in Figure 1 and Figure 2, demonstrate that the system energy exhibits symmetric behavior centered at $15 \text{ \AA}$ (half the $30 \text{ \AA}$ box size) under periodic boundary conditions, confirming correct computation of periodicity. For forces, beyond $15 \text{ \AA}$ separation, the force magnitude aligns with expectations where one particle effectively interacts with the image of the other from the opposite box boundary. The FPGA-calculated energy shows an root mean square error (RMSE) of $3.5 \times 10^{-6} \text{ kcal/mol}$ compared to CPU/GPU results, confirming identical computational outcomes. These results validate that our MD implementation correctly computes LJ interactions on FPGAs.

Building upon successful validation of the Lennard-Jones (LJ) potential in two-particle systems, this test extends characterization to a 15-atom Argon cluster under periodic boundary conditions ($30 \text{ \AA}$ cubic box). Parameters derived from the CHARMM force field ($\sigma = 3.8220 \text{ \AA}$, $\varepsilon = 0.238065 \text{ kcal/mol}$) were implemented with a $12 \text{ \AA}$ non-bonded interaction cutoff. Five distinct structural configurations were tested across FPGA, GPU, and CPU platforms using identical hardware setups as previously reported. The LJ potential energy calculations (Table 5) demonstrate $<5\%$ inter-platform RMS deviation across all test cases, confirming FPGA implementation accuracy. This cross-platform consistency validates the FPGA’s computational precision in handling LJ interactions through optimized parallel architectures and memory bandwidth utilization.

After validating the energy calculations of the protein system, we further conducted MD simulations on the FPGA platform to verify its capability for authentic simulation tasks. As illustrated in Figures 3, the test system employed the DHFR model, a benchmark protein consisting of 159 amino acid residues and 2489 atoms commonly used in computational studies.

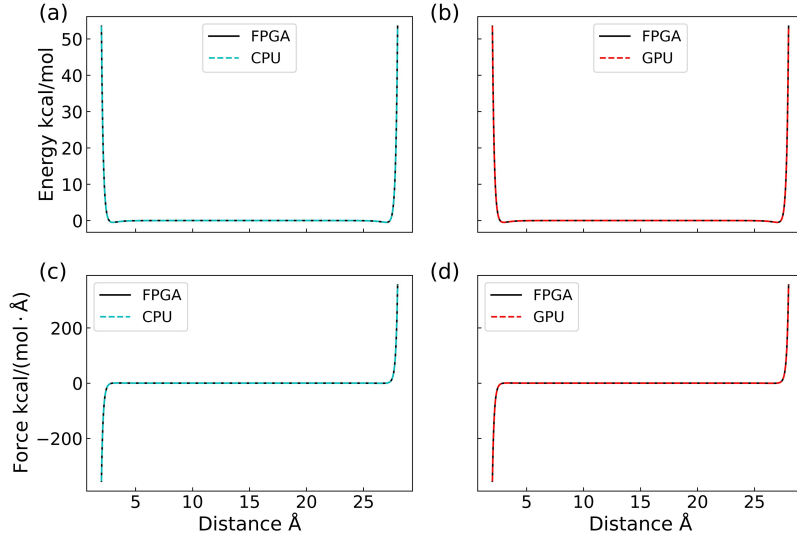


Figure 1: Comparison of two-particle LJ system energy and single-particle forces as functions of inter-particle distance in 30 Å periodic box: (a) Total energy comparison between FPGA and CPU calculations. (b) Total energy comparison between FPGA and GPU calculations. (c) Single-particle force comparison between FPGA and CPU calculations. (d) Single-particle force comparison between FPGA and GPU calculations.

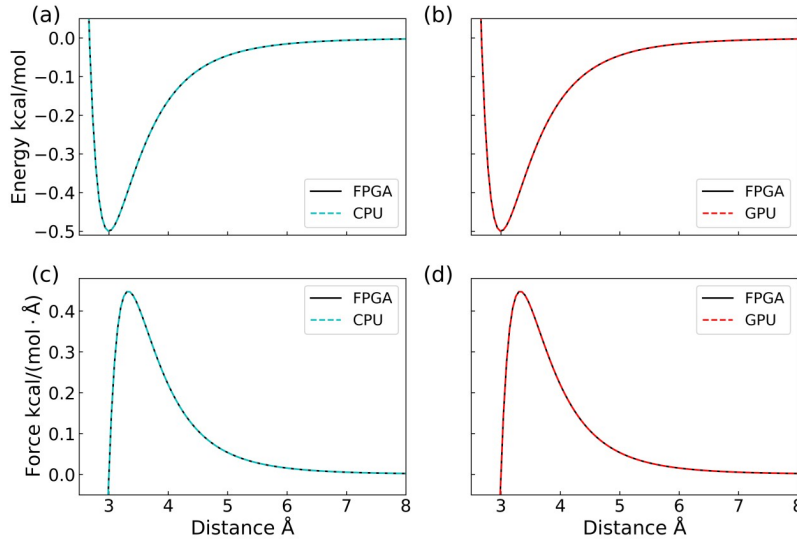


Figure 2: Detailed comparison of two-particle LJ system energy and single-particle forces from 2.5 Å to 8 Å in 30 Å periodic box: (a) Total energy comparison (FPGA vs. CPU). (b) Total energy comparison (FPGA vs. GPU). (c) Single-particle force comparison (FPGA vs. CPU). (d) Single-particle force comparison (FPGA vs. GPU).

The simulation parameters included a time step of 2 fs and 3D-FFT with two spatial resolutions: $32 \times 32 \times 32$ and $64 \times 64 \times 64$. Comparative analyses were performed against CPU and GPU implementations. During the 3D-FFT process, systematic errors were observed, primarily attributed to two factors: (1) The FPGA's native FFT IP core only supports matrix dimensions as powers of two (e.g., 32-bit or 64-bit inputs), and (2) The FFT v9.0 IP core offers both 32-bit floating-point and fixed-point modes, with distinct resource consumption trade-offs.

Table 5: Total LJ interaction energy calculated for a system of fifteen argon atoms on five structures.

	Potential Energy (kJ/mol)		
	FPGA	CPU	GPU
Frame1	0.923459	0.922628	0.922628
Frame2	-2.184186	-2.182222	-2.182222
Frame3	-3.207323	-3.204440	-3.204439
Frame4	-1.750265	-1.748692	-1.748693
Frame5	-1.624073	-1.622612	-1.622612

Table 6: Energy results of a system using AMBER ff99SB force field under 5 different structures.

		Potential Energy (kJ/mol)		
		FPGA	CPU	GPU
Frame1	bond	2087.662245	2087.862775	2087.860086
	angle	5433.357112	5433.900774	5433.902874
	torsion	6907.129606	6908.421362	6908.423872
	nonbonded	-312340.490691	-312345.961500	-312345.563700
	total	-297910.382283	-297915.776600	-297915.376900
Frame2	bond	1907.251060	1907.111822	1907.108081
	angle	5511.006220	5511.585466	5511.584221
	torsion	6927.941477	6928.677819	6928.680307
	nonbonded	-310742.773636	-310748.284000	-310747.930400
	total	-296395.409418	-296400.908900	-296400.557800
Frame3	bond	1928.601210	1928.751160	1928.746112
	angle	5310.363338	5310.518108	5310.524570
	torsion	6755.237077	6755.889797	6755.893233
	nonbonded	-308428.479107	-308434.056000	-308433.703100
	total	-294433.338691	-294438.896900	-294438.539300
Frame4	bond	1922.334000	1922.482161	1922.481287
	angle	5359.920702	5360.031861	5360.025824
	torsion	7027.404300	7026.701621	7026.704579
	nonbonded	-310148.639490	-310154.021000	-310153.664500
	total	-295839.426078	-295844.805400	-295844.452800
Frame5	bond	1947.426859	1947.278810	1947.275611
	angle	5342.720827	5342.273262	5342.276176
	torsion	7001.926522	7001.272750	7001.275782
	nonbonded	-310775.911073	-310781.230700	-310780.861400
	total	-296485.091978	-296490.405800	-296490.033800

DISCUSSION

In MD calculations, there is a large amount of intermediate data directly related to particle information, so data storage requires an independent interval. DDR has a single read-write channel, while HBM supports 32 independent channels, greatly improving data throughput for particle computing read/write storage. Multi-channel computation enhances particle read and write efficiency to external memory, but the on-chip resource consumption of the read/write driver must be considered. We use a progress proofreading module to supervise multi-channel data for orderly data access.

In terms of computing performance, the current 250 MHz main frequency of the FPGA is on par

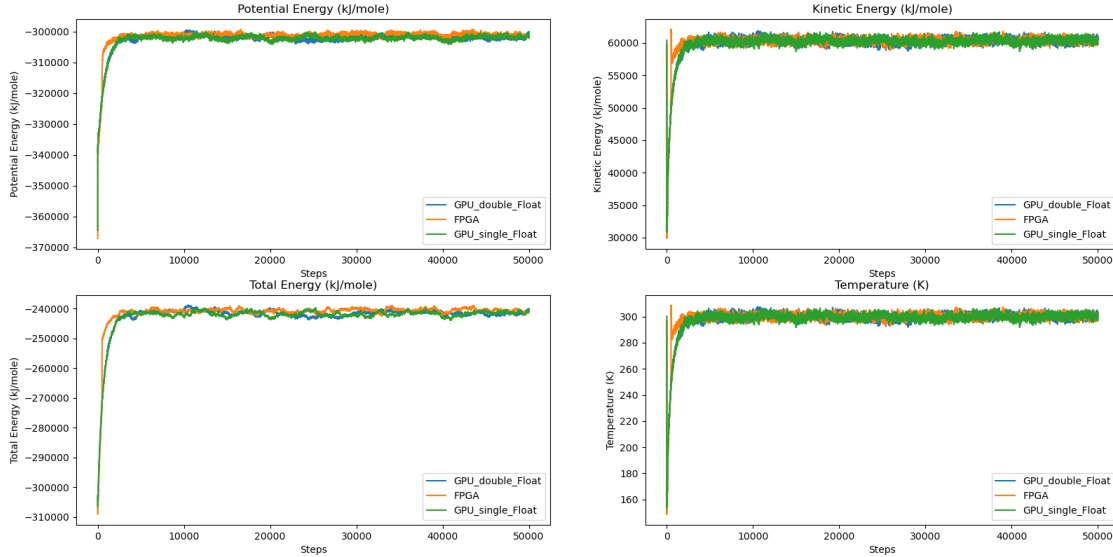


Figure 3: DHFR protein system simulation results, PME algorithm using 32 point 3D FFT transformation. Compared with the FPGA accelerated version, CPU and GPU versions, the data calculation results are basically the same.

with that of the CPU, but it is far from reaching its theoretical performance limit. This is mainly due to the fixed point precision constraint of the Fourier transform IP core and the redundant logic levels in the pipeline design. For example, in the 3D Fourier transform, the selection of 32 bit fixed point numbers balances resource consumption and error tolerance (error < 0.0001), at cost of computing throughput. In the future, a mixed precision architecture can be explored, introducing low precision calculations in non critical paths to increase the main frequency. Additionally, although the hardware/software co design scheme for covalent bond calculation has verified its correctness, the data round trip transmission (e.g., the SHAKE algorithm remains on the CPU side) introduces additional latency. In the future, it is necessary to explore a fully hardware based force field calculation pipeline design or optimize the heterogeneous interface to reduce communication overhead.

From the verification results, the FPGA shows highly consistent results with the CPU/GPU in the energy calculations of the LJ potential, water box model, and DHFR, with the RMSE as low as 10^{-6} kcal/mol, which proves the numerical stability and functional correctness of the hardware implementation. However, in the speed test, the FPGA’s computing speed for the largest system (17,615 particles) is only 0.0287 ns/day, revealing the deficiency in large scale parallel computing capabilities. This is related to both the main frequency limitation and the bandwidth bottleneck between the computing units and the storage subsystem. It is recommended that in subsequent optimizations, the FPGA’s main frequency should be prioritized to be increased to the target frequency of 500 MHz, the data path should be reconfigured to reduce the logic delay of the critical path, and a NoC should be introduced to optimize the data interaction efficiency between multiple modules.

Overall, the current design verifies the feasibility of FPGAs in MD calculations. However, to achieve an energy efficiency ratio comparable to that of GPUs, performance needs to be further improved through iterations of the hardware architecture, adaptation of algorithms to hardware, and compilation optimizations.

RESOURCE AVAILABILITY	217
Lead contact	218
Requests for further information and resources should be directed to and will be fulfilled by the lead contact, Sally White (s.white@university.edu).	219 220
Materials availability	221
Plasmids generated in this study have been deposited to [Addgene, name and catalog number or unique identifier].	222 223
Data and code availability	224
- Small-molecule crystallography data have been deposited at CCDC under the database identifier ABCDEF and are publicly available as of the date of publication. ¹H NMR spectra have been deposited at Mendeley under the DOI 10.12345/a1b2c3d4e5f6g7.1 and are publicly available as of the date of publication. All other data reported in this paper will be shared by the lead contact upon request. - All original code has been deposited at Zenodo under the DOI 10.56789/a1b2c3d4e5f6g7.1 and is publicly available as of the date of publication.³⁶ - Any additional information required to reanalyze the data reported in this paper is available from the lead contact upon request.	225 226 227 228 229 230 231 232 233
ACKNOWLEDGMENTS	234
This work was funded by [FUNDER] via grant [GRANT NO.]. The authors thank all members of the lab for their support.	235 236
AUTHOR CONTRIBUTIONS	237
Conceptualization, S.C.P. and S.Y.W.; methodology, A.B., S.C.P., and S.Y.W.; investigation, M.E., A.N.V., N.A.V., S.C.P., and S.Y.W.; writing—original draft, S.C.P. and S.Y.W.; writing—review & editing, S.C.P. and S.Y.W.; funding acquisition, S.C.P. and S.Y.W.; resources, M.E.V and C.K.B.; supervision, A.B., N.L.W., and A.A.D.	238 239 240 241
DECLARATION OF INTERESTS	242
S.Y.W. is an employee and shareholder of COMPANY. M.E.V. is a founder of COMPANY and a member of its scientific advisory board.	243 244
DECLARATION OF GENERATIVE AI AND AI-ASSISTED TECHNOLOGIES	245 246
During the preparation of this work, the author(s) used [NAME OF TOOL OR SERVICE] in order to [REASON]. After using this tool or service, the author(s) reviewed and edited the content as	247 248

needed and take(s) full responsibility for the content of the publication. 249

SUPPLEMENTAL INFORMATION INDEX 250

Figures S1-S5 and their legends in a PDF 251

Table S1. A descriptive title for an Excel file that was too large to appear in the PDF 252

Table S2. Another descriptive title for a different Excel file 253

Data S1. Raw data on x, y, and z 254

1. Qian, R., Xue, J., Xu, Y., and Huang, J. (2024). Alchemical transformations and beyond: Recent advances and real-world applications of free energy calculations in drug discovery. *Journal of Chemical Information and Modeling* 64, 7214–7237. URL: <https://doi.org/10.1021/acs.jcim.4c01024>. doi: 10.1021/acs.jcim.4c01024. arXiv:<https://doi.org/10.1021/acs.jcim.4c01024>. PMID: 39360948.
2. York, D.M. (2023). Modern alchemical free energy methods for drug discovery explained. *ACS Physical Chemistry Au* 3, 478–491. URL: <https://doi.org/10.1021/acspphyschemau.3c00033>. doi: 10.1021/acspphyschemau.3c00033. arXiv:<https://doi.org/10.1021/acspphyschemau.3c00033>.
3. De Vivo, M., Masetti, M., Bottegoni, G., and Cavalli, A. (2016). Role of molecular dynamics and related methods in drug discovery. *Journal of Medicinal Chemistry* 59, 4035–4061. URL: <https://doi.org/10.1021/acs.jmedchem.5b01684>. doi: 10.1021/acs.jmedchem.5b01684. arXiv:<https://doi.org/10.1021/acs.jmedchem.5b01684>. PMID: 26807648.
4. Donaldson, L. (2013). Tying crystals in knots. *Materials Today* 16, 360. URL: <https://www.sciencedirect.com/science/article/pii/S1369702113003064>. doi: <https://doi.org/10.1016/j.mattod.2013.09.009>.
5. E, W., and Vanden-Eijnden, E. (2010). Transition-path theory and path-finding algorithms for the study of rare events. *Annual Review of Physical Chemistry* 61, 391–420.
6. Xie, T., Song, Z., and Huang, J. (2024). Conditioned protein structure prediction. *PRX Life* 2, 043001. URL: <https://link.aps.org/doi/10.1103/PRXLife.2.043001>. doi: 10.1103/PRXLife.2.043001.
7. Kumar, G., and Mishra, A., Radha Ramanand Verma (2022). Introduction to molecular dynamics simulations. In A. Verma, S. Mavinkere Rangappa, S. Ogata, and S. Siengchin, eds. *Forcefields for Atomistic-Scale Simulations: Materials and Applications*. Singapore: Springer Nature Singapore. ISBN 978-981-19-3092-8 pp. 1–19.
8. Hwang, W., Austin, S.L., Blondel, A., Boittier, E.D., Boresch, S., Buck, M., Buckner, J., Caffisch, A., Chang, H.T., Cheng, X., Choi, Y.K., Chu, J.W., Crowley, M.F., Cui, Q., Damjanovic, A., Deng, Y., Devereux, M., Ding, X., Feig, M.F., Gao, J., Glowacki, D.R., Gonzales, J.E.I., Hamaneh, M.B., Harder, E.D., Hayes, R.L., Huang, J., Huang, Y., Hudson, P.S., Im, W., Islam, S.M., Jiang, W., Jones, M.R., Käser, S., Kearns, F.L., Kern, N.R., Klauda, J.B., Lazaridis, T., Lee, J., Lemkul, J.A., Liu, X., Luo, Y., MacKerell, A.D.J., Major, D.T., Meuwly, M., Nam, K., Nilsson, L., Ovchinnikov, V., Paci, E., Park, S., Pastor, R.W., Pittman, A.R., Post, C.B., Prasad, S., Pu, J., Qi, Y., Rathinavelan, T., Roe, D.R., Roux, B., Rowley, C.N., Shen, J., Simmonett, A.C., Sodt, A.J., Töpfer, K., Upadhyay, M., van der Vaart, A., Vazquez-Salazar, L.I., Venable, R.M., Warrensford, L.C., Woodcock, H.L., Wu, Y., Brooks, C.L.I., Brooks, B.R., and Karplus, M. (2024). Charmm at 45: Enhancements in accessibility, functionality, and speed. *The Journal of Physical Chemistry B* 128, 9976–10042. URL: <https://doi.org/10.1021/acs.jpcb.4c04100>. doi: 10.1021/acs.jpcb.4c04100. arXiv:<https://doi.org/10.1021/acs.jpcb.4c04100>. PMID: 39303207.
9. Salomon-Ferrer, R., Götz, A.W., Poole, D., Le Grand, S., and Walker, R.C. (2013). Routine microsecond molecular dynamics simulations with amber on gpus. 2. explicit solvent particle mesh ewald. *Journal of Chemical Theory and Computation*

- 9, 3878–3888. URL: <https://doi.org/10.1021/ct400314y>. doi: 10.1021/ct400314y. 299
arXiv:<https://doi.org/10.1021/ct400314y>. PMID: 26592383. 300
10. Phillips, J.C., Hardy, D.J., Maia, J.D.C., Stone, J.E., Ribeiro, J.V., Bernardi, R.C., Buch, 301
R., Fiorin, G., Hénin, J., Jiang, W., McGreevy, R., Melo, M.C.R., Radak, B.K., Skeel, R.D., 302
Singharoy, A., Wang, Y., Roux, B., Aksimentiev, A., Luthey-Schulten, Z., Kalé, L.V., Schulten, 303
K., Chipot, C., and Tajkhorshid, E. (2020). Scalable molecular dynamics on cpu and gpu 304
architectures with namd. *The Journal of Chemical Physics* 153, 044130. URL: <https://doi.org/10.1063/5.0014475>. doi: 10.1063/5.0014475. 305
306
11. Eastman, P., Swails, J., Chodera, J.D., McGibbon, R.T., Zhao, Y., Beauchamp, K.A., Wang, 307
L.P., Simmonett, A.C., Harrigan, M.P., Stern, C.D., Wiewiora, R.P., Brooks, B.R., and Pande, 308
V.S. (2017). Openmm 7: Rapid development of high performance algorithms for molecular dy- 309
namics. *PLOS Computational Biology* 13, 1–17. URL: <https://doi.org/10.1371/journal.pcbi.1005659>. 310
doi: 10.1371/journal.pcbi.1005659. 311
12. Abraham, M.J., Murtola, T., Schulz, R., Páll, S., Smith, J.C., Hess, B., and Lindahl, E. (2015). 312
Gromacs: High performance molecular simulations through multi-level parallelism from laptops 313
to supercomputers. *SoftwareX* 1-2, 19–25. URL: <https://www.sciencedirect.com/science/article/pii/S2352711015000059>. doi: <https://doi.org/10.1016/j.softx.2015.06.001>. 314
315
13. Jones, D., Allen, J.E., Yang, Y., Drew Bennett, W.F., Gokhale, M., Moshiri, N., and Rosing, 316
T.S. (2022). Accelerators for classical molecular dynamics simulations of biomolecules. *Journal* 317
of Chemical Theory and Computation 18, 4047–4069. 318
14. Narumi, T., Yasuoka, K., Taiji, M., Zerbetto, F., and Höfinger, S. (2011). Fast calculation of 319
electrostatic potentials on the gpu or the asic md-grape-3. *The Computer Journal* 54, 1181– 320
1187. 321
15. Morimoto, G., Koyama, Y.M., Zhang, H., Komatsu, T.S., Ohno, Y., Nishida, K., Ohmura, 322
I., Koyama, H., and Taijit, M. (2021). Hardware acceleration of tensor-structured multilevel 323
ewald summation method on mdgrape-4a, a special-purpose computer system for molecular 324
dynamics simulations. In *SC21: International Conference for High Performance Computing,* 325
Networking, Storage and Analysis. pp. 1–15. doi: 10.1145/3458817.3476190. 326
16. Shaw, D.E., Deneroff, M.M., Dror, R.O., Kuskin, J.S., Larson, R.H., Salmon, J.K., Young, C., 327
Batson, B., Bowers, K.J., Chao, J.C., Eastwood, M.P., Gagliardo, J., Grossman, J.P., Ho, C.R., 328
Ierardi, D.J., Kolossváry, I., Klepeis, J.L., Layman, T., McLeavey, C., Moraes, M.A., Mueller, 329
R., Priest, E.C., Shan, Y., Spengler, J., Theobald, M., Towles, B., and Wang, S.C. (2008). 330
Anton, a special-purpose machine for molecular dynamics simulation. *Commun. ACM* 51, 91– 331
97. URL: <https://doi.org/10.1145/1364782.1364802>. doi: 10.1145/1364782.1364802. 332
17. Shaw, D.E., Grossman, J., Bank, J.A., Batson, B., Butts, J.A., Chao, J.C., Deneroff, M.M., 333
Dror, R.O., Even, A., Fenton, C.H., Forte, A., Gagliardo, J., Gill, G., Greskamp, B., Ho, C.R., 334
Ierardi, D.J., Iserovich, L., Kuskin, J.S., Larson, R.H., Layman, T., Lee, L.S., Lerer, A.K., 335
Li, C., Killebrew, D., Mackenzie, K.M., Mok, S.Y.H., Moraes, M.A., Mueller, R., Nociolo, 336
L.J., Peticolas, J.L., Quan, T., Ramot, D., Salmon, J.K., Scarpazza, D.P., Schafer, U.B., 337
Siddique, N., Snyder, C.W., Spengler, J., Tang, P.T.P., Theobald, M., Toma, H., Towles, 338
B., Vitale, B., Wang, S.C., and Young, C. (2014). Anton 2: Raising the bar for performance 339
and programmability in a special-purpose molecular dynamics supercomputer. In *SC '14:* 340
Proceedings of the International Conference for High Performance Computing, Networking, 341
Storage and Analysis. pp. 41–53. doi: 10.1109/SC.2014.9. 342

18. Shaw, D.E., Adams, P.J., Azaria, A., Bank, J.A., Batson, B., Bell, A., Bergdorf, M., Bhatt, J., Butts, J.A., Correia, T., Dirks, R.M., Dror, R.O., Eastwood, M.P., Edwards, B., Even, A., Feldmann, P., Fenn, M., Fenton, C.H., Forte, A., Gagliardo, J., Gill, G., Gorlatova, M., Greskamp, B., Grossman, J., Gullingsrud, J., Harper, A., Hasenplaugh, W., Heily, M., Heshmat, B.C., Hunt, J., Ierardi, D.J., Iserovich, L., Jackson, B.L., Johnson, N.P., Kirk, M.M., Klepeis, J.L., Kuskin, J.S., Mackenzie, K.M., Mader, R.J., McGowen, R., McLaughlin, A., Moraes, M.A., Nasr, M.H., Nociolo, L.J., O'Donnell, L., Parker, A., Peticolas, J.L., Pocina, G., Predescu, C., Quan, T., Salmon, J.K., Schwink, C., Shim, K.S., Siddique, N., Spengler, J., Szalay, T., Tabladillo, R., Tartler, R., Taube, A.G., Theobald, M., Towles, B., Vick, W., Wang, S.C., Wazlowski, M., Weingarten, M.J., Williams, J.M., and Yuh, K.A. (2021). Anton 3: twenty microseconds of molecular dynamics simulation before lunch. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. SC '21. New York, NY, USA: Association for Computing Machinery. ISBN 9781450384421.
19. George, A., Lam, H., and Stitt, G. (2011). Novo-g: At the forefront of scalable reconfigurable supercomputing. *Computing in Science Engineering* 13, 82–86.
20. George, A.D., Herbordt, M.C., Lam, H., Lawande, A.G., Sheng, J., and Yang, C. (2016). Novo-g : Large-scale reconfigurable computing with direct and programmable interconnects. In 2016 IEEE High Performance Extreme Computing Conference (HPEC). pp. 1–7.
21. Cabrera, A.M., Yucesan, Y.A., Liu, F.Y., Blokland, W., and Vetter, J.S. (2023). Errant beam detection using the amd versal acap and vitis ai. In 2023 IEEE High Performance Extreme Computing Conference (HPEC). pp. 1–6. doi: 10.1109/HPEC58863.2023.10363622.
22. Kilts, S. (2007). *Advanced FPGA Design: Architecture, Implementation, and Optimization*. Wiley-IEEE Press. ISBN 0470054379.
23. Waidyasooriya, H.M., Hariyama, M., and Kasahara, K. (2016). Architecture of an fpga accelerator for molecular dynamics simulation using opencl. In 2016 IEEE/ACIS 15th International Conference on Computer and Information Science (ICIS). pp. 1–5. doi: 10.1109/ICIS.2016.7550743.
24. Stewart, L.C., Pascoe, C., Davis, E., Sherman, B.W., Herbordt, M., and Sachdeva, V. (2021). Particle mesh ewald for molecular dynamics in opencl on an fpga cluster. In 2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM). pp. 270–270.
25. Yang, Y., Long, W., Kannan, R., and Prasanna, V.K. (2023). Fpga acceleration of rotation in homomorphic encryption using dynamic data layout. In 2023 33rd International Conference on Field-Programmable Logic and Applications (FPL). pp. 174–181. doi: 10.1109/FPL60245.2023.00032.
26. Choi, M., Xu, X., and Moroz, V. (2019). Modeling performance and thermal induced reliability issues of a 3nm finfet logic chip operation in a fan-out and a flip-chip packages. In 2019 18th IEEE Intersociety Conference on Thermal and Thermomechanical Phenomena in Electronic Systems (ITherm). pp. 107–112. doi: 10.1109/ITHERM.2019.8757442.
27. Khan, U., Owen, H., and Hughes, J. (1993). Fpga architectures for asic hardware emulators. In Sixth Annual IEEE International ASIC Conference and Exhibit. pp. 336–340. doi: 10.1109/ASIC.1993.410733.

28. Yang, C., Geng, T., Wang, T., Patel, R., Xiong, Q., Sanaullah, A., Sheng, J., Lin, C., Sachdeva, V., Sherman, W., and Herbordt, M.C. (2019). Fully Integrated On-FPGA Molecular Dynamics Simulations. *arXiv e-prints* pp. arXiv:1905.05359. doi: 10.48550/arXiv.1905.05359. arXiv:1905.05359.
29. Fujita, N., Kobayashi, R., Yamaguchi, Y., and Boku, T. (2021). Hbm2 memory system for hpc applications on an fpga. In 2021 IEEE International Conference on Cluster Computing (CLUSTER). pp. 783–786. doi: 10.1109/Cluster48925.2021.00116.
30. Choi, Y.K., Chi, Y., Wang, J., and Cong, J. (2020). Flash: Fast, parallel, and accurate simulator for hls. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 4828–4841. doi: 10.1109/TCAD.2020.2970597.
31. Curreri, J., Koehler, S., Holland, B., and George, A.D. (2008). Performance analysis with high-level languages for high-performance reconfigurable computing. In 2008 16th International Symposium on Field-Programmable Custom Computing Machines. pp. 23–30. doi: 10.1109/FCCM.2008.18.
32. Varma, B.S.C., Paul, K., and Balakrishnan, M. (2013). Accelerating 3d-fft using hard embedded blocks in fpgas. In 2013 26th International Conference on VLSI Design and 2013 12th International Conference on Embedded Systems. pp. 92–97. doi: 10.1109/VLSID.2013.169.
33. Sheng, J., Yang, C., Sanaullah, A., Papamichael, M., Caulfield, A., and Herbordt, M.C. (2017). Hpc on fpga clouds: 3d ffts and implications for molecular dynamics. In 2017 27th International Conference on Field Programmable Logic and Applications (FPL). pp. 1–4.
34. Eastman, P., Swails, J., Chodera, J.D., McGibbon, R.T., Zhao, Y., Beauchamp, K.A., Wang, L.P., Simmonett, A.C., Harrigan, M.P., Stern, C.D., Wiewiora, R.P., Brooks, B.R., and Pande, V.S. (2017). Openmm 7: Rapid development of high performance algorithms for molecular dynamics. *PLOS Computational Biology* 13, 1–17. URL: <https://doi.org/10.1371/journal.pcbi.1005659>. doi: 10.1371/journal.pcbi.1005659.
35. Eastman, P., Galvelis, R., Peláez, R.P., Abreu, C.R.A., Farr, S.E., Gallicchio, E., Gorenko, A., Henry, M.M., Hu, F., Huang, J., Krämer, A., Michel, J., Mitchell, J.A., Pande, V.S., Rodrigues, J.P., Rodriguez-Guerra, J., Simmonett, A.C., Singh, S., Swails, J., Turner, P., Wang, Y., Zhang, I., Chodera, J.D., De Fabritiis, G., and Markland, T.E. (2024). Openmm 8: Molecular dynamics simulation with machine learning potentials. *The Journal of Physical Chemistry B* 128, 109–116. arXiv:<https://doi.org/10.1021/acs.jpcb.3c06662>. PMID: 38154096.
36. Rizos, G. (2023). Code for the article “Propagating Variational Model Uncertainty for Bioacoustic Call Label Smoothing”. Zenodo. <https://doi.org/10.5281/zenodo.10253149>.
37. Rapaport, D.C. (2004). *The Art of Molecular Dynamics Simulation*. 2 ed.. Cambridge University Press.
38. Vanommeslaeghe, K., and MacKerell, A. (2015). Charmm additive and polarizable force fields for biophysics and computer-aided drug design. *Biochimica et Biophysica Acta (BBA) - General Subjects* 1850, 861–871. URL: <https://www.sciencedirect.com/science/article/pii/S0304416514002736>. doi: <https://doi.org/10.1016/j.bbagen.2014.08.004>. Recent developments of molecular dynamics.

39. Jayaraman, S., Zhang, B., and Prasanna, V. (2022). Hypersort: High-performance parallel sorting on hbm-enabled fpga. In 2022 International Conference on Field-Programmable Technology (ICFPT). pp. 1–11. doi: 10.1109/ICFPT56656.2022.9974209.
40. (2019). Ieee standard for floating-point arithmetic. IEEE Std 754-2019 (Revision of IEEE 754-2008) pp. 1–84. doi: 10.1109/IEEESTD.2019.8766229.
41. Overchick, E.R., and Abramov, B. (2014). Designing of single precision floating point dsp co-processor. In 2014 IEEE 28th Convention of Electrical & Electronics Engineers in Israel (IEEEI). pp. 1–5.
42. Villarreal, J., and Najjar, W.A. (2008). Compiled hardware acceleration of molecular dynamics code. In 2008 International Conference on Field Programmable Logic and Applications. pp. 667–670.
43. Yang, C., Geng, T., Wang, T., Lin, C., Sheng, J., Sachdeva, V., Sherman, W., and Herbordt, M. (2019). Molecular dynamics range-limited force evaluation optimized for fpgas. In 2019 IEEE 30th International Conference on Application-specific Systems, Architectures and Processors (ASAP) vol. 2160-052X. pp. 263–271.
44. Darden, T.A., York, D.M., and Pedersen, L.G. (1993). Particle mesh ewald: An $n \cdot \log(N)$ method for ewald sums in large systems. Journal of Chemical Physics 98, 10089–10092. URL: <https://api.semanticscholar.org/CorpusID:55356282>.
45. Duan, Z.H., and Krasny, R. (2000). An ewald summation based multipole method. Journal of Chemical Physics 113, 3492–3495.
46. Aatish, K., and Zhang, B. (2013). Three dimensional fast fourier transform cuda implementation. URL: <https://api.semanticscholar.org/CorpusID:17582708>.
47. Gu, Y., VanCourt, T., and Herbordt, M. (2005). Accelerating molecular dynamics simulations with configurable circuits. In International Conference on Field Programmable Logic and Applications, 2005. pp. 475–480.

Hardware

452

In this work, the Inspur F37X FPGA accelerator card used in this work was selected for its advanced programmable logic architecture, offering high flexibility and precise customization to address specific algorithmic requirements of the project. This adaptability effectively overcomes performance bottlenecks inherent in traditional computing architectures, enabling breakthroughs in technical barriers and fostering innovative research pathways. The accelerator card leverages the Xilinx Virtex UltraScale+ VU37P chip, integrating 8 GB HBM2 high-bandwidth memory with a bandwidth of 460 GB/s, delivering 28.1 TOPS INT8 compute performance at < 75 W power consumption.[?] This configuration enables high-speed scientific computing while maintaining energy efficiency, outperforming CPUs in scenarios requiring low latency and high throughput.

453

454

455

456

457

458

459

460

461

Basic algorithms of classical MD and force field

462

In this work, we divide a classical MD program into four parts³⁷:

463

1. Parse the specified simulation conditions, such as initial temperature, particle number, and time step.
2. Initialize positions and velocities of the particles.
3. Compute the forces acting on all particles.
4. Integrate Newton's equations to update the velocities and positions of all particles.

464

465

466

467

468

Repeat steps 3 and 4 until the user specified simulation step is met. In classical MD simulations, the force acting on each atom is often calculated using force fields which will be discussed in the next two subsections.

469

470

471

Bonded force

472

Most force fields model interactions between particles using bonded and nonbonded terms:

473

$$\mathbf{F}_{\text{total}} = \mathbf{F}_{\text{bonded}} + \mathbf{F}_{\text{nonbonded}} \quad (1)$$

,where the bonded force consists of bond, angle, dihedral and improper terms:

474

$$\mathbf{F}_{\text{bonded}} = \mathbf{F}_{\text{bonds}} + \mathbf{F}_{\text{angles}} + \mathbf{F}_{\text{dihedrals}} + \mathbf{F}_{\text{impropers}} \quad (2)$$

Most bonded terms are modeled by harmonic potentials, except the dihedrals are modeled using a summation of cosine functions.³⁸ Because the bonded terms are only calculated when bonds exist between atoms, their computational cost can be regarded as $\mathcal{O}(N)$.

475

476

477

478

Nonbonded force

479

The non-bonded terms often consist of van der Waals (vdW) and Coulomb terms:

480

$$\mathbf{F}_i^{\text{LJ}} = \sum_{j \neq i} \frac{\varepsilon_{ab}}{\sigma_{ab}^2} \left\{ 12 \left(\frac{\sigma_{ab}}{|\mathbf{r}_{ji}|} \right)^{14} - 6 \left(\frac{\sigma_{ab}}{|\mathbf{r}_{ji}|} \right)^8 \right\} \mathbf{r}_{ji} \quad (3)$$

$$\mathbf{F}_i^{\text{Coulomb}} = \frac{q_i}{4\pi\epsilon} \sum_{j \neq i} \left\{ \frac{1}{|\mathbf{r}_{ji}|} \right\}^3 \mathbf{r}_{ji} \quad (4)$$

where, the ϵ_{ab} and σ_{ab} are parameters of particles. Because the nonbonded force need to be calculated between nearly all pair-wise particles, the computational cost is $\mathcal{O}(N^2)$. Thus the bottom neck in MD is the calculation of nonbonded terms.

Because the decay of coulomb force is slow with distance. it is always calculated with the PME algorithm in a perodical system to handle the long range Coulomb forces. Assuming that the unit cell is square, the Coulomb potential on atom i can be expressed as Eq 4. The algorithm utilizes spherical Gaussian positive and negative charges with the same distribution to partition E^{Coulomb} into two parts in the real and the reciprocal spaces. The electrostatic potential energy of the simulated system under PBC is expressed as:

$$E^{\text{Coulomb}} = \frac{1}{4\epsilon\pi} \sum_n \sum_{i>j}^N \sum_{j=1}^N \frac{q_i q_j}{|r_{ij} + nL|} \quad (5)$$

The simulation box is a square of length L , where the number of charged particles is N . r_{ij} is the distance between particle i and particle j , and ϵ is the vacuum dielectric constant. The simulation lattice extends indefinitely in the control according to the PBC, and n is an integer.

$$E^{\text{Coulomb}} = E_{\text{dir}} + E_{\text{rec}} + E_{\text{corr}} \quad (6)$$

The Gaussian distribution function is introduced into the Ewald summation algorithm and consists of three parts after being processed by the PME algorithm:

$$E_{\text{dir}} = \frac{1}{2} \sum_n \sum_{i,j=1}^N \frac{q_i q_j \operatorname{erf}(\beta |\mathbf{r}_j - \mathbf{r}_i + \mathbf{n}|)}{|\mathbf{r}_j - \mathbf{r}_i + \mathbf{n}|} \quad (7)$$

$$E_{\text{rec}} = \frac{1}{2\pi V} \sum_{\mathbf{m} \neq 0} \frac{\exp(-\pi^2 \mathbf{m}^2 / \beta^2)}{\mathbf{m}^2} S(\mathbf{m}) S(-\mathbf{m}) \quad (8)$$

$$E_{\text{corr}} = -\frac{1}{2} \sum_{(i,j) \in M} \frac{q_i q_j \operatorname{erf}(\beta |\mathbf{r}_i - \mathbf{r}_j|)}{|\mathbf{r}_i - \mathbf{r}_j|} - \frac{\beta}{\sqrt{\pi}} \sum_{i=1}^N q_i^2 \quad (9)$$

,where the β is called Ewald's coefficient, which regulates the speed of convergence of real space and Fourier space. The larger its value, the faster the real space converges and the slower the Fourier space converges and vice versa. The erf is the Gaussian error function.

To improve computational efficiency, the finite range force in the direct space sometimes uses a proximity list lookup method which obtains the spatial relationships between particles using their coordinates and distances, and stores this data in a neighborhood list. When affected by short range forces, a particle only interacts with its neighboring particles. As the distance increases, all

particles in the neighborhood are treated as neighbors, and their coordinates are updated to reconstruct interrelationships in a new step. The neighbor list is constructed based on each particle's position within a specific region, assuming that the interaction force between particles is truncated at a radius r_{cut} . Each particle contains a list of potential neighboring particles, which serves as input for the next computational cell. In addition, for periodical system, each simulation cell has 26 adjacent cells in the whole space. According to Newton's third law, to reduce repeated calculations, only the particle pairs in the 13 neighboring cells in the upper half of the center cell are filtered and calculated, as shown in Figure 4.

Verlet integrator algorithm

Given the force acting on each particle, the velocity and position is updated by the integrator. Commonly used integrators include the Verlet algorithm and its variants, such as the Velocity Verlet and Leap-frog methods. Using the acceleration and position at time t_0 , the position ($\mathbf{r}$) and velocity ($\mathbf{v}$) of next step are obtained:

$$\mathbf{r}_i(t_0 + \Delta t) = 2\mathbf{r}_i(t_0) - \mathbf{r}_i(t_0 - \Delta t) + (\Delta t^2/m_i)\mathbf{F}_i(t_0) \quad (10)$$

$$\mathbf{v}_i(t_0) = (\mathbf{r}_i(t_0 + \Delta t) - \mathbf{r}_i(t_0 - \Delta t))/2\Delta t \quad (11)$$

The Verlet algorithm requires a motion update calculation for each particle (i) in the system, and the calculation requires a two step difference formulation. It needs to utilize the position information of the particle at the previous moment $\mathbf{r}_i(t_0 - \Delta t)$ and the acceleration $\mathbf{F}_i(t_0)/m_i$ and position information at the current moment $\mathbf{r}_i(t_0)$. In implementing the algorithm, the position and acceleration of the current moment and the position of the previous moment need to be stored.

1 Proposed architecture

Based on the MD algorithm, this paper proposes an FPGA architecture for accelerating MD simulations. As shown in Figure 5, in the range limited (RL) mode, the data processing flow is as follows: First, the CPU preprocesses the MD simulation data, assigns a unique identifier to each particle, and divides the original data according to the cell serial number, thereby obtaining each cell. Then, the particle information is bound to the corresponding cell serial number. The side length of the cell is determined by the cutoff distance, and the number of cells depends on the size of the simulation box in which the simulation dataset is located. Each cell contains not only the information of the particles inside it but also the information of the neighboring cells, and these pieces of information together form a data table stored in the memory.

Each particle occupies one row of storage space. The starting position of the data unit (cell) can be directly located through the cell serial number, and the length of the data unit is determined according to the maximum number of particles with an appropriate margin reserved. The exclusion information data table containing the information of the bonded particle pairs is stored in the memory. The data table structure will exclude the particle information in pairs, and the number of excluded particles and their storage addresses will be configured in the register for use by the relevant modules. The neighboring force calculation uses the interpolation table and coefficient table prepared by the CPU and configured in the RAM of the FPGA. After the configuration is completed, the CPU initiates the prefetch of the particle data unit to perform subsequent operations,

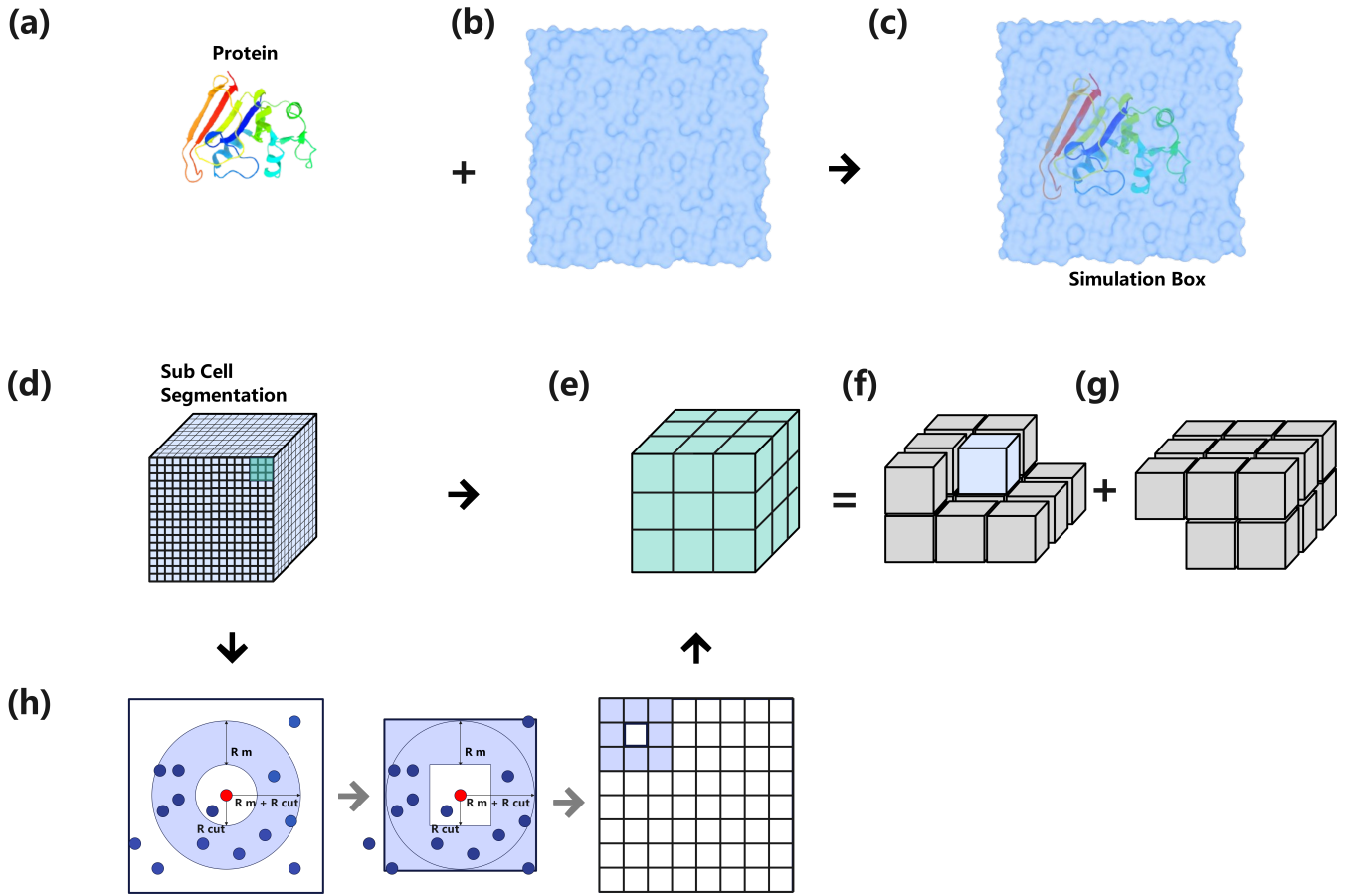


Figure 4: (a) The protein structure of DHFR . (c) A cubic simulated water box. (d) A cubic simulation box. (e) Home Cell and the surrounding 26 Neighbor cells of which the upper and lower two are divided into "upper hemisphere" and "lower hemisphere". (f) Upper hemisphere. (g) lower hemisphere . (h) According to the Verlet algorithm, the center particle and the surrounding particles can be divided into equal sized squares according to the values of R_m parameter and R_{cut} off

and the parallel particle reading module provides the particle pair information for the calculation. Subsequently, the multi-channel calculation screening module processes each data unit as the main cell. It obtains particle information from the main cell and 13 adjacent cells, constructing particle pairs for distance calculation. Specifically, each particle in the main cell is paired with one particle from each of the 13 surrounding cells. The module then filters out particle pairs whose distances fall within the cutoff for further calculations. Interpolation table coefficients are derived from the squared distances of the selected particle pairs and their type information, with these coefficients, based on charge information, used for calculating electrostatic and short-range forces. All parameters for the limited-range force calculation are stored in the on-chip RAM of the FPGA and transmitted to the particle distance judgment module, ensuring that only particles within the cutoff proceed to the next stage of force calculation. The short range force calculation module monitors this process, updating the force calculation result data table and overseeing the calculation progress.

Data is read and written using HBM memory

The HBM2 stores data units in a structured manner, and the configuration of each data unit contains multiple information layers.^{29,39} The main row designates key basic metadata, such as the

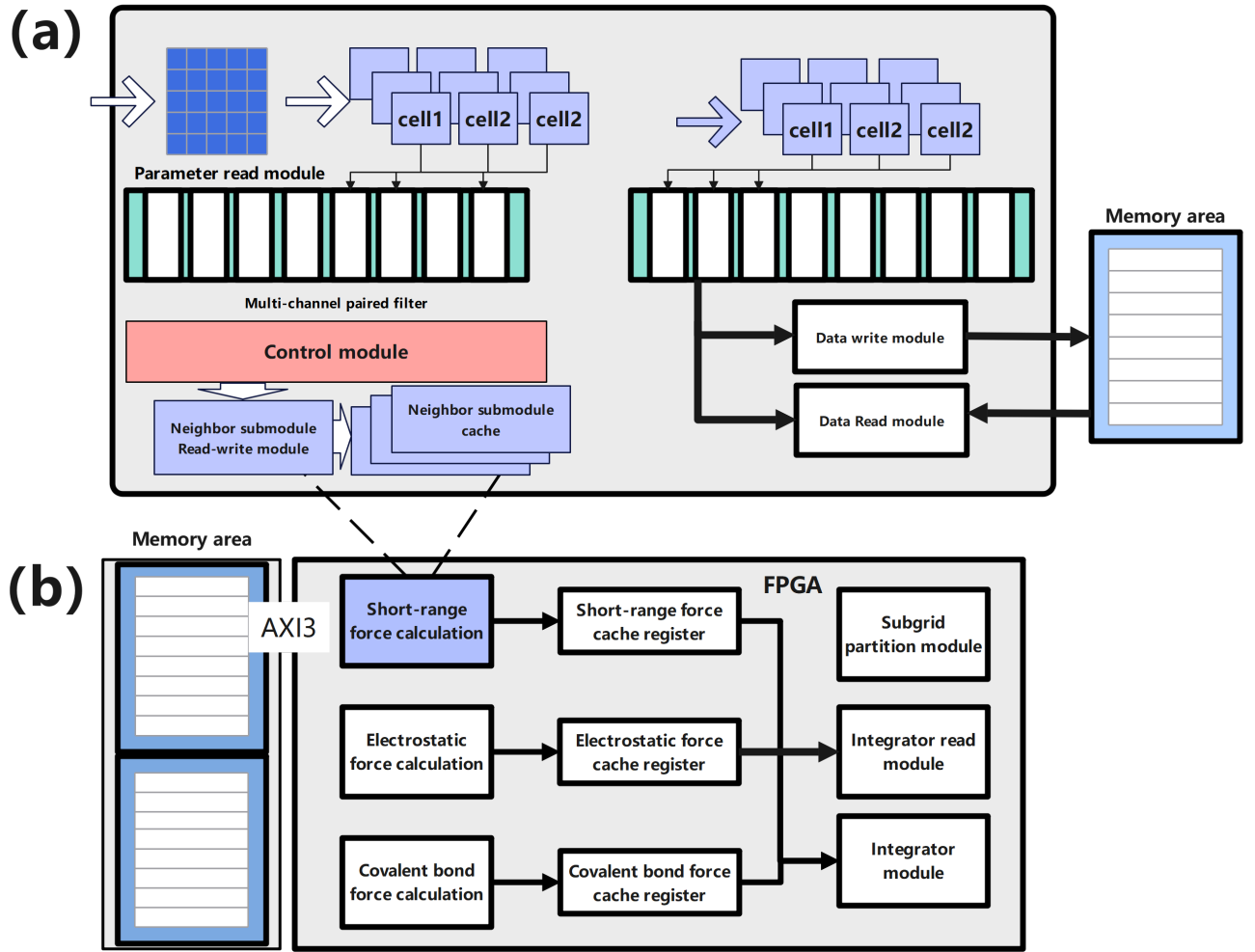


Figure 5: Block diagram of the implementation range Limited force calculation module. (b) A single FPGA acceleration card, the user logic resources MD hardware computation is mainly divided into range limited force , long range force and covalent bonding force three kinds of processing units. (a) The whole RL module is composed in each sub-module of the frame.

cell serial number, particle charge, number of particles, storage offset address of the particle information, and parity bits. These pieces of information lay the foundation for subsequent processing. Subsequently, each quadrant uses the serialized identifier and the HBM address offset of the adjacent units to record the adjacency relationship in an orderly manner. Each spatial dimension is presented in a continuous 8-byte block, making the data association clearly traceable. The memory allocation follows a strict alignment protocol. For incomplete rows, padding is carried out to meet the requirements of the complete row boundary. The particle-specific serial numbers are stored sequentially in the continuous memory segments, and the established alignment rules are maintained through terminal padding to ensure the standardization of data storage. Such a hierarchical arrangement not only ensures the integrity of the memory architecture but also enables efficient data retrieval. When caching data, multiple channels can simultaneously read data in the manner shown in Figure 6(a) and Figure 6(b), either at the same address or different addresses.

Particle Table: Home Cell - Neighbor Cell Storage Relationship

The data units adopt a specialized row allocation strategy to accommodate a single particle. During the preprocessing stage, the size parameters of the data units are determined according to the rule

of increasing the maximum expected number of particles within the specified safety range. Through the hierarchical addressing mechanism, any data unit can be directly accessed by virtue of its unique serial number and mapped to the precise memory coordinates. Structural omissions are made for the particle items, resulting in a specialized memory topology. This topology is elaborated in detail in the figure, and these elements serve as parameters that can be directly compared and used by the calculation modules of the remaining 13 neighboring cells. For the calculation of non-bonding forces, the interpolation coefficients and lookup tables are pre-calculated by the CPU and loaded into the dedicated on-chip RAM resources within the FPGA structure. As for the covalent bond information that does not require calculation from the exclusion data source, such as the hydrogen bond force, etc., the identifiers of the special bonded particles are stored in pairs in the shared memory area, so as to ensure low-latency access during the execution time of the calculation process.

The storage relationship between Home Cell and Neighbor Cell in the particle information storage table.

The particle information storage architecture establishes a dual-layer hierarchy between Home-Cell and NeighborCell structures operating on 512-bit data buses, where non-functional regions undergo zero-padding optimization to maintain address alignment and facilitate efficient lookup operations as shown in Figure 7(b). This unified storage schema integrates 7 distinct functional segments characterized by predefined bit allocations: initial addressing utilizes a lookup code for rapid data classification, directing processing pipelines to appropriate computational modules based on encoded metadata. Serial number assignment implements a globally unique addressing scheme across all simulation entities, while spatial coordinates embed precise 3D positioning parameters compliant with cartesian coordinate systems. Particle identifiers adopt globally unique numbering conventions to track individual entities throughout simulation lifecycles. The system employs a hybrid addressing paradigm where hierarchical indexing of cell relationships coexists with direct particle access patterns, supported by register-mapped exclusion tables that dynamically configure FPGA resources for force calculation acceleration. This structured approach enables deterministic memory access latencies while preserving compatibility with parallel processing architectures.

Data Structure of the Home Cell in the Particle Information Storage Table

The particle information repository employs a fixed-length encoding scheme within 512-bit Home-Cell structures, adhering to memory alignment constraints through zero-padding strategies that optimize address resolution efficiency (shown in Figure 7(c)). This standardized format segments data into 5 predefined fields with dedicated bit allocations, initiating with a machine-readable type identifier that directs processing workflows to designated arithmetic units. Each particle's physical attributes—including scalar charge magnitudes, vector velocity components, and 3D Cartesian coordinates are encoded using IEEE 754-compliant 32-bit floating-point representations for numerical stability⁴⁰.

Parallel processing requirements necessitate dual-tier memory management: precomputed pairwise interaction data for the first 13 nearest NeighborCells resides in on-chip registers for immediate access, while remaining 13 required neighbor interactions are staged in 256-bit wide global memory buffers as shown in Figure 7(c). This partitioning balances latency-critical intra-cell computations with bandwidth-intensive inter-cell calculations, facilitated by FPGA-optimized neighbor index register files that minimize off-chip memory transactions. The hybrid storage methodology simultaneously satisfies deterministic timing constraints for core physics kernels while maintaining scalable data handling capacity for large-scale particle systems.

Result Accumulation: Home Cell with 13 Neighbor Cells

622

The particle-to-force caching and partial summation module implements latency-optimized data reuse by sequentially capturing force calculation results between a primary cell's particle and its intra-cell counterparts across successive clock cycles. This approach consolidates redundant force components into interim storage buffers, substantially reducing redundant data transfers while preserving computational coherence. Critical to this mechanism is the hierarchical aggregation logic that systematically associates force contributions with their originating source particle via unique serial number indexing, enabling parallelized accumulation of vector forces within dedicated on-chip registers. Resultant composite force vectors are then serialized into contiguous memory locations according to source particle identifiers, forming structured force caches that maintain strict alignment with downstream processing requirements. The RL (reaction) force buffer adopts a linear addressing scheme where entries align directly with particle enumeration, ensuring deterministic memory access patterns essential for maintaining synchronization during large-scale N-body simulations. This structured methodology simultaneously minimizes off-chip bandwidth consumption and maximizes FPGA resource utilization through register-file optimized data structures.

637

The Storage Table of the Remaining 13 Neighbor Cells

638

The data caching during the particle calculation process adopts a unified hierarchical storage architecture. Its 512-bit data frame structure is shown in Figure 8 (b), which is composed of two functional rows, namely the index row and the data row. The index row undertakes the task of encoding key operation parameters, and sets fields such as the type classification flag and the particle identifier through dedicated bits. Among them, the buffer capacity parameter is predefined as twice the observed maximum number of particles during the preprocessing stage. The valid entry counter is initially set to zero, and there is also a 16-byte segment checksum for error detection. The buffer management follows a two-stage protocol to handle data insertion and calculation. The dynamic memory offset is determined according to the valid entry count. The counter is incremented after each write operation, and at the same time, the checksum of the first 16 bytes of the index row is regenerated. The dual-buffer design endows the system with the ability to resist overflow during peak calculation cycles. The FPGA logic not only supports hardware-accelerated initialization, that is, resetting all counters and flags, but also supports the reconfiguration of the simulated reset managed by software. The IEEE 754 single-precision floating-point format is adopted to process the velocity, position, and charge components, ensuring the numerical stability in the parallel processing pipeline. Compatibility with the FPGA-based DMA engine is maintained, thereby improving the data transfer efficiency from the host to the device.

656

Generation of Interpolation and Coefficient Tables.

657

As shown in Figure 9, the range-limited force calculation assumes that particle i and particle j have already undergone the particle pair screening module. only particle pairs that meet certain criteria (e.g., within a cutoff distance) are processed through the DSP direct calculation pipeline for efficient force computation. This approach ensures higher numerical accuracy compared to fixed-point or lower-precision alternatives. However, since FP32 arithmetic consumes more DSP resources than integer or reduced-precision operations,⁴¹ it imposes a trade-off between precision and computational efficiency.

664

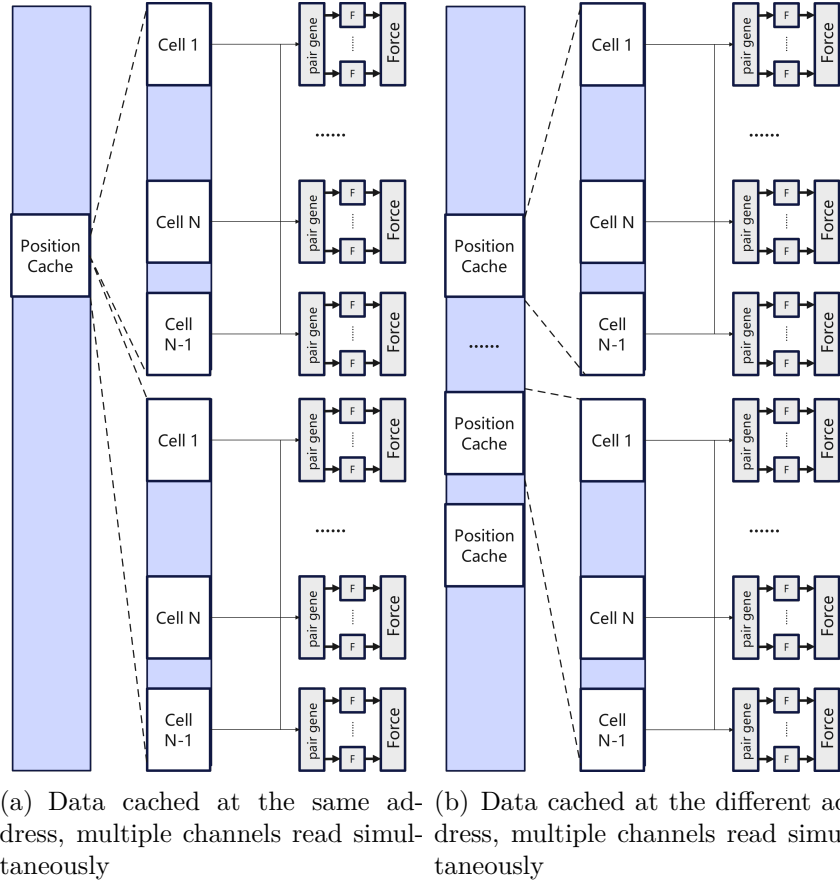


Figure 6: Data cached at the same address, multiple channels read simultaneously; Data cached at the different address, multiple channels read simultaneously

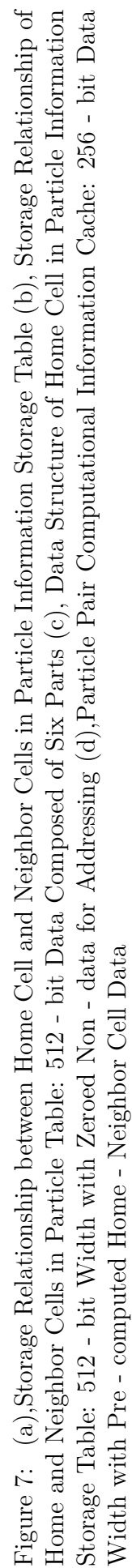
In existing studies, interpolation lookup tables are utilized to replace the traditional direct calculation method, thereby conserving resources.^{28,42,43} And forces are calculated by:

$$\begin{aligned}
 F^{\text{RL}} &= \frac{A_{ab}}{r_{ij}^{13}} + \frac{B_{ab}}{r_{ij}^7} + \frac{Q_{ab}}{r_{ij}^2} \\
 &= r_{ij} \cdot \left(\frac{A_{ab}}{r_{ij}^{14}} + \frac{B_{ab}}{r_{ij}^8} + \frac{Q_{ab}}{r_{ij}^3} \right)
 \end{aligned} \tag{12}$$

, where $A_{ab} = 48\varepsilon_{ab}\sigma_{ab}^{12}$, $B_{ab} = -12\varepsilon_{ab}\sigma_{ab}^6$, and $Q_{ab} = q^a q^b / 4\pi\varepsilon$.

In this design, r_{ij}^{-14} , r_{ij}^{-8} and r_{ij}^{-3} are used in the formula of range limited force. In this design, r_{ij}^2 is used as the output, and all the interpolating functions of the above three functions are created, which are reduced to low-order monic polynomials and then brought into r_{ij}^2 to compute the original values, thus simplifying the computation process and reducing the consumption of DSP resources. The generation of the interpolating function should not only reduce the computational complexity, but also ensure that the accuracy of the computation meets the design requirements. The r_{ij}^{-14} , r_{ij}^{-8} and r_{ij}^{-3} can be approximated using a multi-order interpolation function. The performance of first-order, second-order, and third-order interpolations was compared respectively.

By establishing a reasonable sampling interval, the derivative of the target function can be numerically approximated at discrete points within the interval. This allows the derivation of a fitting function valid over the selected range. The following formula is a set of equations for the 2nd-order



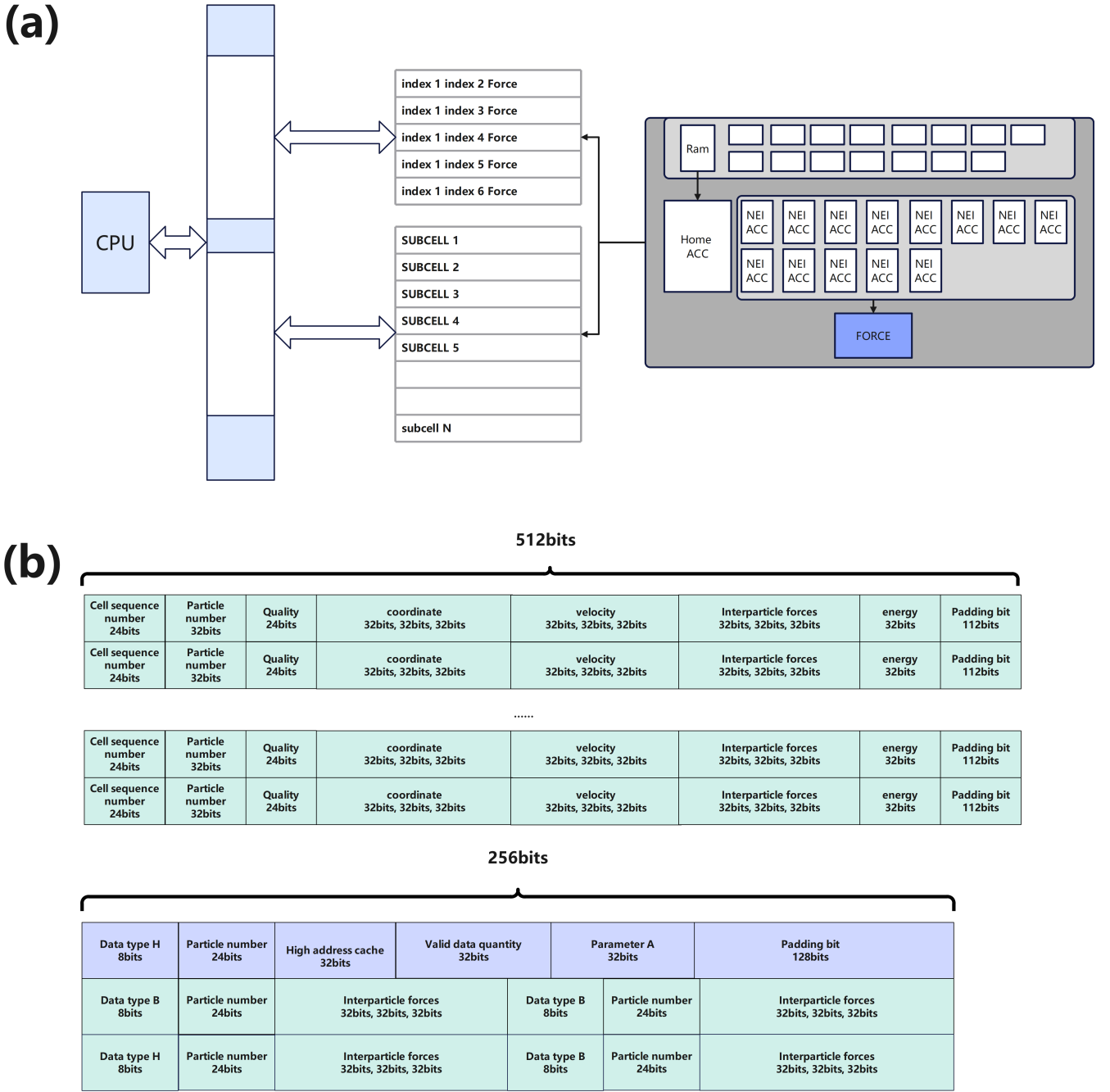


Figure 8: (a), the Home Cell and the remaining 13 neighbor cells accumulate the results, and the data needs to be filtered a lot before the particles in the corresponding cell can calculate the final short-range force. (b), the block diagram of the storage structure of the RL force result buffer, the output of the calculation result of the particle pair, and the storage length of each particle information is 512 bits. Block diagram of the storage structure of the RL force result buffer, the output of the calculation result of the particle pair, the storage length of each particle information is 256bits, and the calculation result of each step is stored in a specific storage space, which is carried by the CPU for the remaining calculation processing.

interpolation function to find the value of the interval $[x_0, x_1]$.

The $x[0.1, 144]$ interval is divided into 8 segments, $[0.1, 0.2]$, $[0.2, 0.4]$, $[0.4, 0.8]$, $[0.8, 1.6]$, $[1.6, 3.2]$, $[3.2, 6.4]$, $[6.4, 16.0]$, $[16, 144]$, of which the first 7 segments, each segment is divided into 16

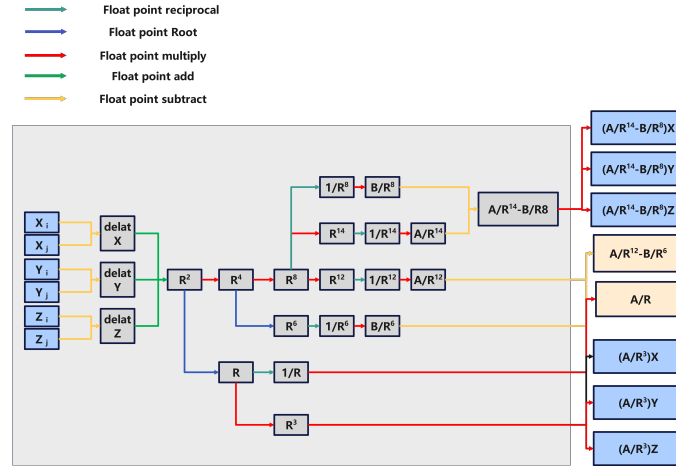


Figure 9: The block diagram of the direct calculation module, assuming that particle i and particle j have gone through the particle screening module, and the short-range force of the particle pairs needs to be calculated within a certain range of action. The particle pairs are calculated directly by the DSP calculation pipeline, and the calculation results are single-precision floating-point numbers, which are calculated by the Xilinx floating-point number calculation unit, which is a relatively high-precision calculation method that consumes a large amount of DSP resources.

small segments, and the last segment, divided into 128 small segments, a total of 240 interpolation functions. Each interpolation function requires 3 coefficients, expressed as single precision floating point numbers. The interpolation table for each function can be fully stored using a 4 KB BRAM. The calculation of the range limited force also requires a table of coefficients for A_{ab} and B_{ab} , which need to be combined two by two for the particle types, so that N particle types will leads to a table with length of N^2 . After analyzing the simulation dataset, there are ≤ 32 independent particle types, and the depth of the coefficient table is ≤ 1024 items.

In the hardware floating point calculation, the root operation always consume a lot of logic resources, so we use r_{ij}^2 to do multi-level multiplication to calculate r_{ij}^{14} and r_{ij}^8 in place of r_{ij}^{13} and r_{ij}^7 , which replaces the floating point root operation. Another lookup table was used to handle Coulomb force, parallel computation does not interfere with each other in the computation time, the data is output to the last step after floating point number multiplication and division, and the corresponding particle parameters are read from the memory, such as A_{ab} , B_{ab} and other parameters, as shown in Figure 10. Because parameters such as A_{ab} and B_{ab} are determined constants in the simulation system, the constant particle pair lookup table can be pre-established, which can reduce the calculation logic and calculation time in FPGA.

The Make Table Module: Its Role in Calculating Process Control

The particle computation oversight framework integrates multiple functional layers to optimize neighbor identification and force aggregation pipelines. As shown in Figure 11, a hierarchical memory architecture manages cell metadata through dynamic matrix structures: a 2D array (Module a) scales with system-wide cell populations to track neighbor indices per home cell, while a 1D auxiliary array (Module b) facilitates rapid index comparison and cross-referencing operations. A parallel comparison engine (Module c) evaluates eligibility criteria across these matrices using hardware-accelerated voting logic, enabling deterministic selection of interacting home cells based on spatial proximity thresholds.

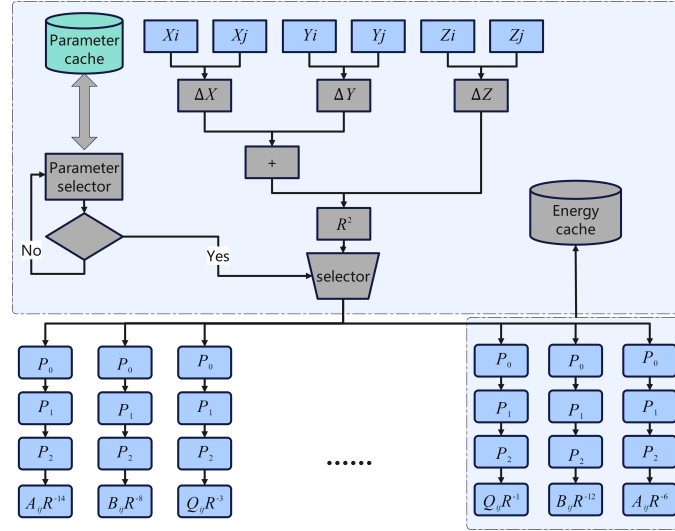


Figure 10: The figure shows the framework diagram of the second-order interpolation table DSP processing flow, the information of each particle is read from the parameter cache, after the arbitration of the parameter selector, the particle pairs go through the distance judgment, and the particles within the computation range enter into the force computation channel, assuming that the particles to be computed are particle i and particle j .

Sequence control is managed through a stateful counter array (Module d) that enforces strict access patterns to prevent memory coherence violations. Data retrieval from external HBM is orchestrated by a pipelined fetch unit (Module e), which retrieves composite particle attribute blocks—each home cell requires 13 cumulative data segments including self and 12 neighbor cells through burst optimized HBM controllers. The external storage subsystem (Module g) employs bank-interleaved addressing to resolve latency bottlenecks arising from concurrent access requests, while a data fusion module (Module f) aggregates force vectors from intra cell and inter-cell interactions using SIMD-aligned accumulation registers. This architecture achieves sub-microsecond neighbor search latencies through optimized matrix compression techniques and pipelined dataflow.

Three-dimensional Fast Fourier transform.

In this work, the computationally intensive three-dimensional(3D) Fast Fourier Transform (FFT) module on an FPGA platform was implemented, while delegating remaining computational tasks to CPUs.^{24,44,45} Data exchange between the CPU and FPGA is facilitated through high speed PCIe interfaces or optical fiber connections.³² In order to accelerate 3D FFT transform in Coulomb force calculations in MD, we developed a parallelized approach leveraging proximity lookup tables for Ewald summation, achieving parallelism comparable to vdW force computations.⁴⁶ The simulation system undergoes cubic or rectangular grid partitioning along X/Y/Z axes, with grid dimensions determined by system boundaries to optimize hardware accelerated data alignment. This design adapts simulation boxes to cubic/rectangular geometries for efficient data tiling. Temporal spatial domain transformations employ optimized FFT algorithms for frequency domain computations, while B-spline interpolation (order=3) maps particle charges to nearest grid points. Hardware implementation leverages OpenMM's reference codebase for kernel optimization, achieving $15 \times$ speedup in Coulomb force calculations compared to CPU only implementations when tested on the Inspur F37X platform. Resource utilization analysis reveals 83% DSP block efficiency and 72% HBM2 bandwidth saturation during protein system simulations (100,000 atoms), demonstrating

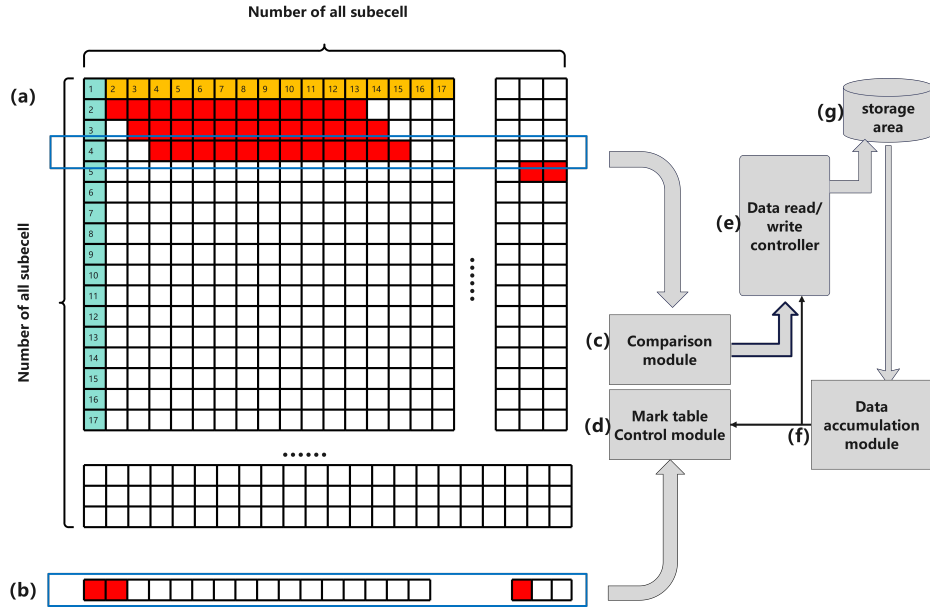


Figure 11: The particle computing progress monitoring module is composed of several modules.

feasibility for large scale MD workloads.

740

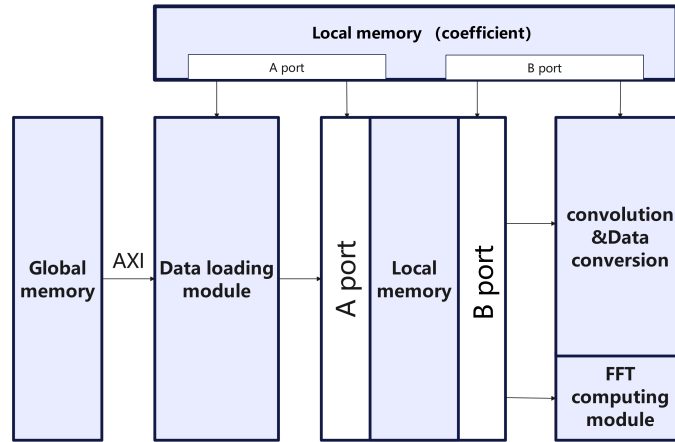


Figure 12: The 3D Fourier transform.

Calculations of the bonded force and energy.

741

Figure 13 shows the cooperation mode between CPU and FPGA in calculating the bonded force, and the data transmission between software and hardware is realized through PCIe.⁴⁷ Each small box in the diagram represents a circuit processing unit. Particle information is derived from user input files, and the structure of the particle information storage table varies according to the different inputs. This data is stored in a specific cache area of HBM2 using DMA technology. In classical MD simulations, bond topology describing how particles are connected remains unchanged during the simulation. The FPGA data reading and writing submodule adopts multiple channels in parallel, and is equipped with HBM2 register, which can support multiple read and write operations simultaneously, therefore is able to read the interaction data between two, three and four bodies respectively. This design greatly improves the storage and access efficiency of larger data blocks.

742

743

744

745

746

747

748

749

750

751

752

The computer flow control circuit is mainly responsible for the management. In the force calculation, the process for the two-body force is relatively simple, thus enabling faster computation. The calculation speed for the three-body force falls between that of the two-body and four-body forces. The four-body force calculation is the slowest, both in terms of the time required and the complexity of the process, constituting the primary bottleneck in the overall calculation. Our covalent bond calculation buffer area, taking into account the varying access frequencies of each calculation channel, incorporates a specialized circuit designed to receive data from different channels.

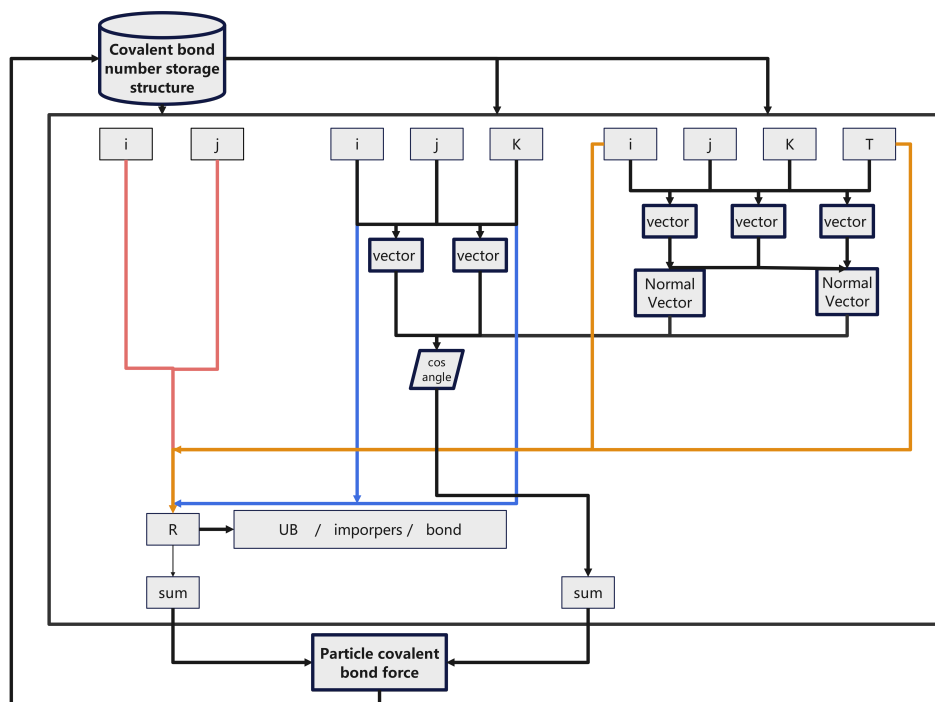


Figure 13: The force of the covalent bond is calculated. These include two-force, three-force, four-force calculations, during which a portion of the IP can be reused.